МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность ПО-13

Выполнил
Бондарчук А.Ю,
студент группы
ПО-13

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«_____k ___2017 г.

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Задание 1.

Условие:

Напишите Python-скрипт, который анализирует активность разработчиков
в указанном публичном GitHub-репозитории и составляет рейтинг самых активных контрибьюторов.

1. Запрашивает у пользователя имя
репозитория (например, pallets/flask или microsoft/vscode).
2. Использует GitHub API для получения списка всех контрибьюторов.
3. Для каждого контрибьютора собирает данные:
- Количество коммитов
- Количество открытых pull requests
- Количество закрытых pull requests
- Количество открытых issues
- Количество закрытых issues
- Дата последней активности
4. Определяет самых активных разработчиков по суммарному вкладу.
5. Строит график вклада разработчиков (matplotlib / seaborn).Код программы

Код:

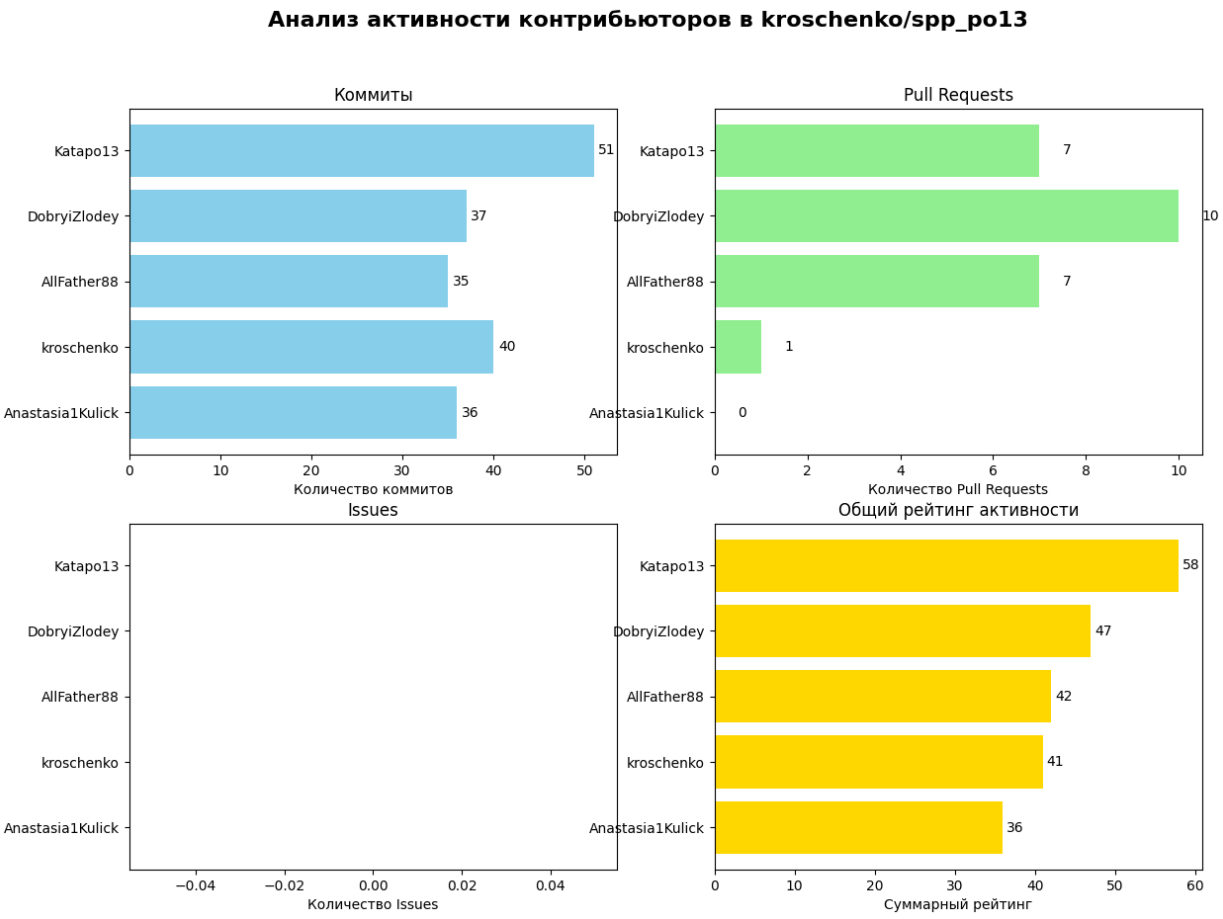
```
import json
from datetime import datetime
import requests
import matplotlib.pyplot as plt
import numpy as np


def load_config():
    with open('config.json', 'r', encoding='utf-8') as f:
        config = json.load(f)
    return config.get('github_token')

GITHUB_TOKEN = load_config()

HEADERS = {
    "Authorization": f"token {GITHUB_TOKEN}",
    "Accept": "application/vnd.github.v3+json",
    "User-Agent": "Python-GitHub-Analyzer/1.0"
}

def get_contributors(repo):
    url = f"https://api.github.com/repos/{repo}/contributors"
    params = {"per_page": 100, "anon": "false"}
    contributors = []

    page = 1
    while True:
        params["page"] = page
```

```
        response = requests.get(url, params=params,headers=HEADERS,
timeout=10)
        if response.status_code != 200:
            print(f"Ошибка при запросе контрибьюторов:
{response.status_code}")
            break

        data = response.json()
        if not data:
            break
        contributors.extend(data)
        page += 1

    return contributors


def get_user_contributions(repo, username):
    search_commits_url = "https://api.github.com/search/commits"
    params_commits = {"q": f"repo:{repo} author:{username}"}

    response_commits = requests.get(search_commits_url,
params=params_commits,headers=HEADERS, timeout=10)
    commits_count = 0
    if response_commits.status_code == 200:
        commits_count = response_commits.json().get("total_count", 0)

    prs_url = "https://api.github.com/search/issues"
    params_pr_open = {"q": f"repo:{repo} type:pr author:{username}
is:open"}
    params_pr_closed = {"q": f"repo:{repo} type:pr author:{username}
is:closed"}

    response_open = requests.get(prs_url,
params=params_pr_open,headers=HEADERS, timeout=10)
    response_closed = requests.get(prs_url,
params=params_pr_closed,headers=HEADERS, timeout=10)

    open_prs = response_open.json().get("total_count", 0)
    closed_prs = response_closed.json().get("total_count", 0)

    issues_url = "https://api.github.com/search/issues"
    params_issues_open = {"q": f"repo:{repo} type:issue
author:{username} is:open"}
    params_issues_closed = {"q": f"repo:{repo} type:issue
author:{username} is:closed"}

    response_issues_open = requests.get(issues_url,
params=params_issues_open,headers=HEADERS, timeout=10)
    response_issues_closed = requests.get(issues_url,
params=params_issues_closed,headers=HEADERS, timeout=10)

    open_issues = response_issues_open.json().get("total_count", 0)
    closed_issues =
response_issues_closed.json().get("total_count",0)

    last_activity = None
    last_commits_url = f"https://api.github.com/repos/{repo}/commits"
    last_params = {"author": username, "per_page": 1}
    last_response = requests.get(last_commits_url,
params=last_params,headers=HEADERS, timeout=10)
    if last_response.status_code == 200 and last_response.json():
        last_commit_date =
```

```
last_response.json()[0]["commit"]["author"]["date"]
        last_activity =
datetime.fromisoformat(last_commit_date.replace('Z', '+00:00'))

    return {
        "commits": commits_count,
        "open_prs": open_prs,
        "closed_prs": closed_prs,
        "open_issues": open_issues,
        "closed_issues": closed_issues,
        "last_activity": last_activity
    }

if __name__ == "__main__":
    repository = input("Введите репозиторий : ")
    print(f"\nАнализируем вклад контрибьюторов в {repository}")

    contributors_list = get_contributors(repository)
    if not contributors_list:
        print("Не удалось получить список контрибьюторов.")

    contributors_data = []
    total_contributors = len(contributors_list)

    for idx, contributor in enumerate(contributors_list):
        user_name = contributor["login"]
        print(f"Обрабатываем контрибьютора {idx +
1}/{total_contributors}: {user_name}")

        user_data = get_user_contributions(repository, user_name)
        user_data["username"] = user_name
        user_data["total_score"] = (user_data["commits"] * 1 +
                                    user_data["open_prs"] * 1 +
                                    user_data["closed_prs"] * 1 +
                                    user_data["open_issues"] * 1 +
                                    user_data["closed_issues"] * 1)
        contributors_data.append(user_data)

    contributors_data.sort(key=lambda x: x["total_score"],
reverse=True)

    print("\nТоп-5 разрабов:")
    for i, contrib in enumerate(contributors_data[:5], 1):
        print(f"{i}. {contrib['username']} - {contrib['commits']}
комитов, "
              f"{contrib['open_prs'] + contrib['closed_prs']} PR, "
              f"{contrib['open_issues'] + contrib['closed_issues']}
issues, "
              f"рейтинг: {contrib['total_score']}")

    usernames = [c["username"] for c in contributors_data]
    commits = [c["commits"] for c in contributors_data]
    prs_total = [c["open_prs"] + c["closed_prs"] for c in
contributors_data]
    issues_total = [c["open_issues"] + c["closed_issues"] for c in
contributors_data]
    total_scores = [c["total_score"] for c in contributors_data]

    _, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(14,
10))
    y_pos = np.arange(len(usernames))
```

```
    ax1.barh(y_pos, commits, color='skyblue')
    ax1.set_yticks(y_pos)
    ax1.set_yticklabels(usernames)
    ax1.invert_yaxis()
    ax1.set_xlabel('Количество коммитов')
    ax1.set_title('Коммиты')
    for idx, v in enumerate(commits):
        ax1.text(v + 0.5, idx, str(v), va='center')

    ax2.barh(y_pos, prs_total, color='lightgreen')
    ax2.set_yticks(y_pos)
    ax2.set_yticklabels(usernames)
    ax2.invert_yaxis()
    ax2.set_xlabel('Количество Pull Requests')
    ax2.set_title('Pull Requests')
    for idx, v in enumerate(prs_total):
        ax2.text(v + 0.5, idx, str(v), va='center')

    ax3.barh(y_pos, issues_total, color='lightcoral')
    ax3.set_yticks(y_pos)
    ax3.set_yticklabels(usernames)
    ax3.invert_yaxis()
    ax3.set_xlabel('Количество Issues')
    ax3.set_title('Issues')
    for idx, v in enumerate(issues_total):
        ax3.text(v + 0.5, idx, str(v), va='center')

    ax4.barh(y_pos, total_scores, color='gold')
    ax4.set_yticks(y_pos)
    ax4.set_yticklabels(usernames)
    ax4.invert_yaxis()
    ax4.set_xlabel('Суммарный рейтинг')
    ax4.set_title('Общий рейтинг активности')
    for idx, v in enumerate(total_scores):
        ax4.text(v + 0.5, idx, str(v), va='center')

    plt.suptitle(f'Анализ активности контрибьюторов в {repository}',
fontsize=16, fontweight='bold')
    plt.savefig("Out.png", dpi=100, bbox_inches='tight')
    plt.tight_layout()
    plt.show()
```

Рисунки с результатами работы программы



Вывод: научился работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API